

VideoVIBE: A Video-Grounded Diagnostic Benchmark for One-Shot Interactive Website Generation

Jiajun Xu¹, Yanghao Zhou^{2‡}, Jingyun Liao³, Yu Bai⁶, Jinxing Zhou⁴,
Chengliang Liu⁷, Changsen Yuan⁵, Bo Wang², Qian Liu⁸

¹University of Technology Sydney, ²Beijing Institute of Technology, ³Hunan University, ⁴OpenNLP Lab
⁵Beijing University of Technology, ⁶Beijing Academy of Artificial Intelligence, ⁷University of Macau, ⁸University of Auckland
[‡]Project Leader.

Abstract

Natural-language-driven “vibe coding” enables the one-shot generation of visually rich and interactive web applications, yet reliable assessment of their quality has not kept pace. Existing evaluations often score isolated artifacts or final task outcomes, offering limited evidence about which failures occur and why. We introduce **VideoVIBE**, a video-grounded benchmark that transforms human-operated webpage recordings into fine-grained diagnostic tasks. It contains approximately 1.7K diagnostic Video QA instances derived from 6,338 verified failures across generated webpages, spanning semantic-logical, visual-motion, structural-temporal, and functional failures. Diagnoses are grounded primarily in recorded presentation and behavior, with webpage source code used as complementary context. We further propose **V2Lens**, a training-free, evidence-grounded multi-agent system that challenges and selectively refines initial video-based diagnoses through targeted visual and source-code verification. Across thirteen closed-source and open-weight Video MLLMs, Gemini-2.5-Flash is the strongest standalone model with a score of 64.54, while V2Lens reaches 71.72, an improvement of 7.18 points. Together, our results show that video-grounded evaluation can move beyond isolated artifacts and aggregate outcomes toward a behaviorally faithful and diagnostically informative account of generated application quality.

Introduction

Coding agents and natural-language-driven “vibe coding” are reshaping software development by enabling users to translate high-level intent directly into executable applications (Lu et al. 2026; Liu et al. 2026; Li et al. 2026; Yang et al. 2024; Wang et al. 2025). Interactive web applications are a particularly important setting because they combine immediate deployability with rich requirements for content, visual design, interaction logic, and application state. In the *one-shot* setting considered in this work, a complete application is generated from a single specification without iterative correction. This capability lowers the barrier to software creation and accelerates rapid prototyping. However, surface completeness does not guarantee behavioral reliability: a visually polished application may still contain inconsistent business rules, broken controls, incorrect state transitions, missing interaction feedback, or failures that emerge only

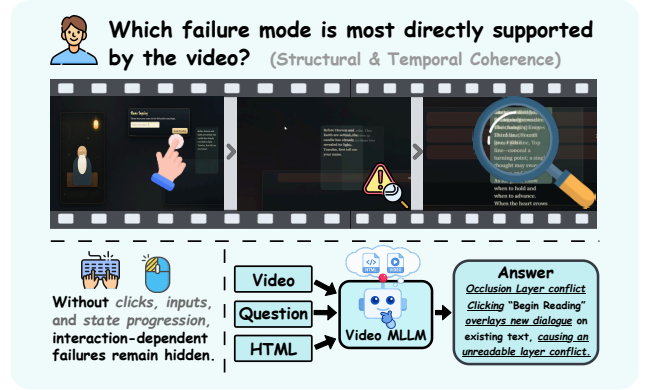


Figure 1: Overview of the VideoVIBE diagnostic task. In this example, a layer conflict becomes visible only after the user enters information and clicks “Begin Reading”, causing a new dialog to overlap existing content. Given the interaction video and diagnostic question, together with complementary webpage source HTML code, a Video MLLM selects the failure mode best supported by the evidence.

after several user actions. As coding agents increasingly participate in application development, identifying such failures becomes essential for determining whether generated software is dependable in actual use.

Evaluating the quality of these applications remains challenging. Existing protocols commonly inspect static artifacts, including screenshots, rendered interfaces, and source code, or summarize application quality through automated tests, workflow success, and coarse-grained human or model judgments (Yun et al. 2024; Lin et al. 2025b; Xiao et al. 2025; Liu et al. 2026; Li et al. 2025a; Zhao et al. 2026; Hong et al. 2026). Although useful, these protocols often assess individual artifacts or final outcomes in isolation, providing limited support for identifying concrete failure cases and distinguishing their underlying modes. Content inconsistencies, poor legibility, and disrupted visual hierarchy may be obscured by aggregate quality scores, while erroneous state transitions, navigation mappings, and broken controls may become apparent only during execution. A comprehensive evaluation should therefore diagnose failures across both rendered interface states and application behavior over time.

Human-operated interaction recordings provide a common source of evidence for these complementary aspects. Individual frames preserve webpage content, layout, typography, visual hierarchy, and motion feedback, while the complete sequence captures user inputs, interface responses, state transitions, navigation behavior, and functional execution (Jang et al. 2024; Lin et al. 2024; Dai et al. 2026; Xu et al. 2026). Fig. 1 illustrates an interaction-dependent example that cannot be identified from the initial rendered state alone. After the user enters the required information and clicks “Begin Reading”, the webpage transitions to a new state in which the newly opened dialog overlaps existing content, making the text unreadable. Other failures, such as inconsistent content, poor contrast, or inappropriate visual hierarchy, can instead be diagnosed directly from rendered frames. Together, frame-level and trajectory-level evidence enable a unified protocol for diagnosing both static presentation defects and interaction-dependent failures.

Based on this formulation, we introduce **VideoVIBE**, a video-grounded diagnostic benchmark for one-shot-generated interactive web applications. VideoVIBE evaluates webpage failures as manifested in recorded application sessions rather than judging source code as a standalone artifact. To ensure diversity and controlled comparison, we apply a shared set of user-goal-oriented prompts across seven representative web-generation systems and record human interactions with the resulting applications. From the retained recordings, we identify 6,338 manually verified failure events and organize them into a hierarchical taxonomy comprising four top-level dimensions, twelve evaluation subdimensions, and thirty fine-grained failure modes. The four dimensions cover *semantic and logical consistency*, *visual and motion fidelity*, *structural and temporal coherence*, and *functional executability*. Representative selection further yields approximately 1.7K diagnostic Video QA instances. Each diagnostic instance asks a model to select the fine-grained failure mode best supported by the evidence. The interaction video provides primary evidence of rendered appearance and observed behavior, while the source code offers complementary implementation-level context for distinguishing similar manifestations with different underlying mechanisms; it is not independently scored. VideoVIBE therefore evaluates evidence-grounded failure-mode discrimination rather than generic source-code inspection, binary failure detection, or end-to-end task success alone.

To improve diagnostic reliability, we propose **V2Lens**, a training-free, evidence-grounded multi-agent system. V2Lens derives an initial video diagnosis, then uses targeted source-code inspection and tool-assisted visual analysis to evaluate alternative hypotheses. Its gated refinement process replaces the initial prediction only when a challenge answer is sufficiently supported and independently confirmed by a reviewer, integrating visual, temporal, and implementation-level evidence while limiting unsupported revisions.

Experiments across thirteen closed-source and open-weight video multimodal models reveal substantial room for improvement. Among standalone models, the closed-source Gemini-2.5-Flash achieves the highest overall score of 64.54, while Qwen3.5-35B-A3B is the strongest open-weight model

with a score of 63.90. No standalone model consistently dominates across all twelve subdimensions, and performance varies across the four failure dimensions, highlighting the difficulty of comprehensive webpage failure diagnosis. Using Gemini-2.5-Flash as its base evaluator, V2Lens improves the overall score from 64.54 to 71.72, a gain of 7.18 points. In summary, our contributions are threefold:

- We introduce **VideoVIBE**, a video-grounded benchmark for diagnosing both static presentation defects and interaction-dependent failures in one-shot-generated web applications.
- We develop an expert-guided taxonomy comprising four top-level dimensions, twelve evaluation subdimensions, and thirty fine-grained failure modes, together with a human-verification protocol for constructing evidence-grounded diagnostic questions.
- We propose **V2Lens**, a training-free multi-agent system that integrates video analysis, source-code verification, tool-assisted evidence collection, and gated refinement, improving the overall diagnostic score by 7.18 points over its standalone base evaluator.

Related Work

Text-to-Web Generation. Driven by rapid advances in large language models (LLMs) and multimodal LLMs (MLLMs), web generation has evolved from static webpage understanding and UI-to-code synthesis to interactive, multi-page, and project-level web application construction (Yun et al. 2024; Lin et al. 2025b; Xiao et al. 2025; Awal et al. 2025). Recent benchmarks evaluate generated applications with complementary protocols: WebGen-Bench executes curated functional test cases using a web-navigation agent (Lu et al. 2026), Vision2Web combines workflow-driven GUI testing with VLM-based visual judging (He et al. 2026), and WebCoderBench aggregates rule-based and LLM-based metrics across multiple quality dimensions (Liu et al. 2026). On the generation side, MM-WebAgent employs hierarchical multimodal planning and iterative reflection, whereas WebGen-R1 uses scaffold-driven reinforcement learning with multimodal rewards to improve project-level website generation (Li et al. 2026; Jiang et al. 2026). These works assess specification satisfaction, functional correctness, visual quality, or aggregate development quality. Their output is typically a task-level score or verdict, rather than a diagnosis of the concrete runtime failure exposed by an interaction trace. In contrast, VideoVIBE takes a human-operated interaction recording and the corresponding HTML implementation as evidence, and evaluates whether a Video MLLM can identify the fine-grained failure mode of an already generated application. This formulation complements end-to-end generation evaluation by making dynamic failure diagnosis the target task.

GUI Evaluation and Video-Grounded Web Modeling. A line of work evaluates interactive systems through agent trajectories and automated judges. AgentRewardBench (Lü et al. 2025) studies whether LLM judges can assess web-agent trajectories, while DiagEval (Hong et al. 2026) uses trajectory-conditioned probes to distinguish evaluator errors from genuine software defects. WebDevJudge (Li et al.

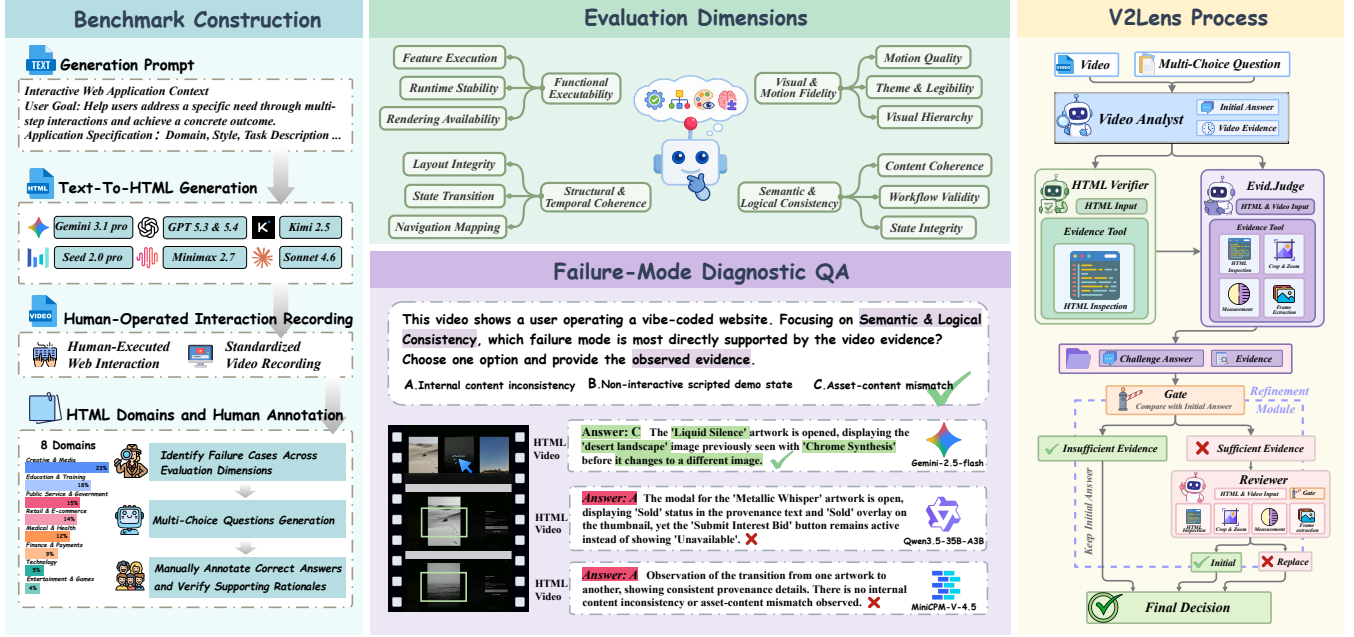


Figure 2: Overview of VideoVIBE and V2Lens. Left: benchmark construction from prompt design and multi-system webpage generation to human-operated recording, failure annotation, and manually verified diagnostic Video QA construction. Middle: the four evaluation dimensions, twelve subdimensions, and representative failure-mode diagnostic instances grounded in interaction videos and webpage source code. Right: the V2Lens evidence-grounded multi-agent process, which combines video analysis, source-code verification, evidence judging, and gated independent review to produce the final diagnosis.

2025a) evaluates LLMs and MLLMs as static and dynamic web-development critics, while computer-use agents have been explored as judges of generative user interfaces (Lin et al. 2025a). VideoWebArena (Jang et al. 2024) evaluates whether agents can use video tutorials to complete web tasks, whereas WebVR (Dai et al. 2026) conditions webpage recreation on screen recordings and assesses reconstruction fidelity with human-aligned visual rubrics. Our VideoVIBE differs from these settings in both its evidence and its output. It does not use video as an instruction for task execution or as a target for webpage reconstruction. Instead, it treats a recording as behavioral evidence of an already generated website and requires models to distinguish confusable failure modes across semantic and logical consistency, visual and motion fidelity, structural and temporal coherence, and functional executability. Its target is therefore neither task success, evaluator reliability, nor reconstruction quality, but the specific failure diagnosis supported by the observed interaction.

VideoVIBE Benchmark

VideoVIBE is a hierarchical diagnostic benchmark for one-shot-generated interactive web applications. Unlike static evaluation based on screenshots or source code alone, it specifically targets failures that emerge as users execute actions, trigger state changes, and complete multi-step workflows. Each benchmark instance combines a human-operated interaction recording with the corresponding webpage source code and requires a model to select the diagnosis best supported by the observed interaction and implementation evidence. Fig. 2 provides an overview of VideoVIBE construc-

tion, its diagnostic task, and the V2Lens process.

Task Definition

Let $\mathcal{D}_i = (V_i, S_i, q_i, \mathcal{A}_i, y_i)$ denote the i -th diagnostic instance, where V_i is a human-operated interaction video, S_i is the corresponding webpage source code, represented as a self-contained HTML document, q_i is a diagnostic question, $\mathcal{A}_i$ is a set of candidate failure modes, and $y_i \in \mathcal{A}_i$ is the ground-truth diagnosis. At evaluation time, a model receives $(V_i, S_i, q_i, \mathcal{A}_i)$ and predicts $\hat{y}_i \in \mathcal{A}_i$.

The two input modalities provide complementary evidence: V_i captures the user action, interface response, and state evolution, whereas S_i provides implementation-level context for distinguishing failures with similar observable manifestations. Each question targets a manually verified failure event within a designated top-level dimension and presents plausible alternative diagnoses. VideoVIBE therefore evaluates fine-grained failure-mode discrimination rather than binary failure detection or end-to-end task-success evaluation.

Failure Taxonomy

To define the diagnostic label space, we develop an expert-guided and empirically grounded failure taxonomy. Domain experts synthesize recurring error patterns in web generation and inspect representative applications produced in practical LLM-assisted vite-coding settings. As illustrated in Fig. 3(a), the resulting hierarchy comprises four top-level dimensions, twelve evaluation subdimensions, and thirty fine-grained failure modes.

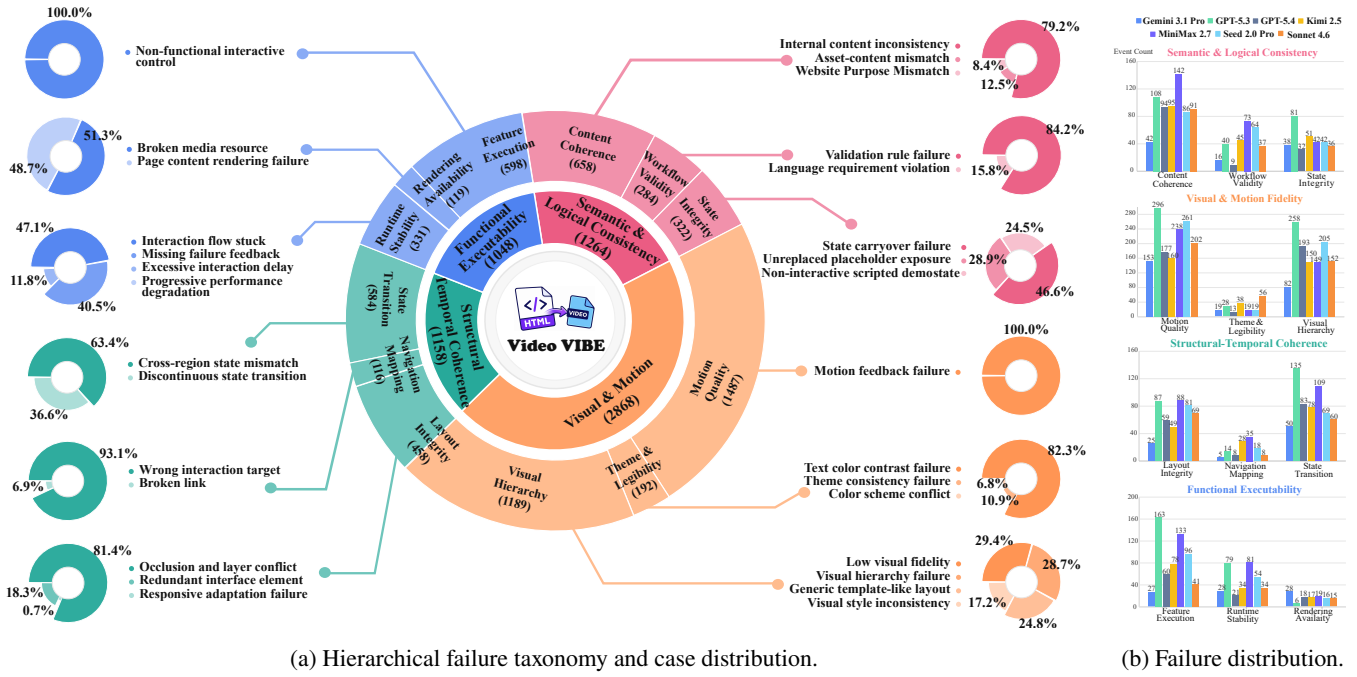


Figure 3: Failure taxonomy and distribution of VideoVIBE. (a) Three-level taxonomy comprising four top-level dimensions, twelve evaluation subdimensions, and thirty fine-grained failure modes. Numbers in parentheses denote verified failure-event counts, while the outer donut charts show the distribution of fine-grained modes within each subdimension. (b) Comparison of verified failure-event counts across twelve evaluation subdimensions for seven webpage-generation systems.

Semantic and Logical Consistency evaluates whether webpage content, business rules, and represented states remain mutually consistent, covering Content Coherence, Workflow Validity, and State Integrity. **Visual and Motion Fidelity** assesses whether visual presentation and motion feedback faithfully communicate user actions and interface responses through Motion Quality, Theme & Legibility, and Visual Hierarchy. **Structural and Temporal Coherence** examines whether layouts, action-target mappings, and successive interface states remain correctly organized throughout interaction, including Layout Integrity, State Transition, and Navigation Mapping. Finally, **Functional Executability** evaluates whether interactive controls, runtime processes, and required resources operate reliably through Feature Execution, Runtime Stability, and Rendering Availability.

Together, these dimensions distinguish what an application communicates, how it presents feedback, how its structure and state evolve, and whether it executes reliably. The thirty fine-grained failure modes provide the diagnostic labels used for question and candidate-answer construction. Fig. 3 presents the complete hierarchy and its failure-case distribution, while detailed explanations of evaluation criteria are provided in the Appendix.

Benchmark Construction and Quality Control

Fig. 4 summarizes the pipeline from webpage generation to human-verified diagnostic questions.

Webpage Generation and Interaction Recording. We construct a shared set of webpage-generation prompts and apply

them across seven webpage-generation models to produce diverse interactive applications under controlled conditions. An agent-assisted audit checks whether each application can be reliably initialized, operated, and recorded. Human operators then follow a standardized interaction protocol that exercises the primary functions, state transitions, and multi-step workflows of each application. Recordings containing at least one observable failure are retained for failure annotation and diagnostic QA construction.

Failure Discovery and Annotation. Automated video analysis identifies candidate anomalies for human review. Annotators then jointly inspect the complete interaction recording and the corresponding webpage source code. A candidate is retained only when its manifestation is observable, reproducible, and unambiguously associated with a fine-grained taxonomy label. For every verified failure event, annotators record its manifestation, supporting evidence, and annotation rationale. Automated tools assist only with candidate discovery, annotation organization, and consistency checking; all final labels and supporting evidence are manually verified.

Diagnostic QA Construction and Verification. We construct diagnostic question-answer pairs from verified failure events, using LLMs only to assist with the initial drafting of questions and candidate options. To reduce redundancy and prevent videos containing many failures from dominating the benchmark, we treat each video-top-level-dimension pair as the unit of representative selection and retain at most one single-choice question for each pair. Multiple annotators independently verify the question wording, candidate options,

ground-truth diagnosis, and consistency with the supporting evidence. Disagreements are resolved through discussion, while unresolved cases are submitted to expert adjudication. Every retained video is associated with at least one manually verified diagnostic question. Annotation details are provided in the Appendix.

Benchmark Coverage & Failure Landscape. VideoVIBE contains 6,338 manually verified failure events with supporting evidence, spanning all twelve subdimensions and thirty fine-grained failure modes. After representative selection, these events yield approximately 1.7K diagnostic question–answer instances.

Fig. 3(b) reports the number of verified failure events for each generator, aggregated from the thirty fine-grained failure modes into twelve evaluation subdimensions. A single video may contain multiple distinct failures, including multiple failures within the same subdimension. Across the seven generators, Motion Quality and Visual Hierarchy exhibit the highest failure counts, followed by Content Coherence, Feature Execution, and State Transition. Their recurrence across multiple systems indicates that these subdimensions represent common challenges in interactive webpage generation rather than model-specific anomalies.

The generators nevertheless exhibit distinct failure profiles. Gemini 3.1 Pro has the fewest failure events, with comparatively fewer failures in Feature Execution, Layout Integrity, and Navigation Mapping. GPT-5.3 and MiniMax 2.7 exhibit higher failure counts but in different areas: GPT-5.3 is more affected by Feature Execution, State Transition, Motion Quality, and Visual Hierarchy, whereas MiniMax 2.7 shows more pronounced weaknesses in Navigation Mapping, Content Coherence, and Workflow Validity. Seed 2.0 Pro exhibits relatively more failures in Motion Quality and Visual Hierarchy, while Claude Sonnet 4.6 shows a more prominent weakness in Theme & Legibility. These results demonstrate that webpage-generation reliability depends not only on static visual quality, but also on maintaining coherent functionality, interaction feedback, state transitions, and application logic.

V2Lens: Evidence-Grounded Multi-Agent Diagnostic System

The diverse and model-specific failure profiles revealed by VideoVIBE underscore the challenge of reliably diagnosing one-shot-generated interactive web applications. Such diagnosis requires relating observable interaction behavior to complementary implementation-level evidence and distinguishing among competing failure hypotheses. To this end, we introduce **V2Lens**, a training-free multi-agent system that performs evidence-grounded verification and conservative answer refinement beyond the one-pass prediction of a standalone multimodal model. V2Lens treats the interaction video as the primary record of observable behavior and uses the webpage source code as complementary implementation-level evidence. It operates entirely at inference time, requiring neither task-specific training nor parameter updates. As illustrated in Fig. 2 (right panel), V2Lens comprises four specialized agents and two deterministic gates organized into

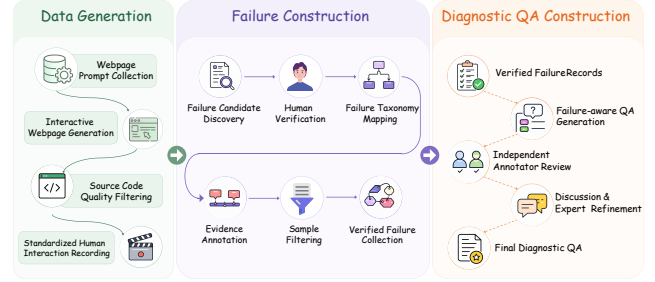


Figure 4: Overview of the VideoVIBE construction and annotation pipeline, from data generation to verified diagnostic QA construction.

three stages: *initial video diagnosis*, *cross-modal evidence verification*, and *gated refinement*. The system first forms a diagnosis from the complete interaction, then searches for evidence that may challenge it, and revises the prediction only when the proposed alternative passes both evidence-sufficiency screening and independent confirmation.

Initial Video Diagnosis. Given an interaction video and its multiple-choice diagnostic question, the *Video Analyst* examines the complete interaction process and produces an initial answer together with supporting video evidence. It identifies the user actions, interface responses, and state changes relevant to the candidate failure modes, thereby establishing the behavioral evidence against which subsequent source-code findings are evaluated.

Cross-Modal Evidence Verification. The *HTML Verifier* performs a focused inspection of the webpage source code. It retrieves implementation fragments relevant to the diagnostic question, including document structure, style rules, event handlers, and state-update logic, and assesses whether they support, contradict, or remain inconclusive with respect to each candidate diagnosis. Its findings provide implementation-level context but do not independently establish that a failure manifested during interaction. The *Evidence Judge* integrates the observations of the Video Analyst, and the findings of the HTML Verifier. When the evidence is insufficient, it may invoke tools for HTML inspection, frame extraction, local-region cropping and enlargement, and visual measurement. These tools enable option-specific examination of both observable behavior and its possible implementation-level causes. The Evidence Judge retains the initial diagnosis unless another candidate is more strongly supported by the collected evidence. When such an alternative exists, it produces a *challenge answer* together with the evidence supporting that challenge.

Gated Refinement. The challenge answer and its supporting evidence are then passed to the *Refinement Module*. The first deterministic gate, the *Evidence Gate*, compares the challenge answer with the initial answer and applies fixed evidence-sufficiency criteria. A challenge is rejected when it lacks direct behavioral support or relies solely on source-code cues without a corresponding manifestation in the interaction video. In this case, V2Lens retains the initial answer. Challenges supported by sufficient evidence are forwarded to the

Table 1: Failure-diagnosis performance (%) of standalone Video MLLMs and V2Lens on VideoVIBE. Best and second-best results in each column are shown in bold and underlined, respectively. Abbreviations: CC = Content Coherence; WV = Workflow Validity; SI = State Integrity; MQ = Motion Quality; TL = Theme & Legibility; VH = Visual Hierarchy; LI = Layout Integrity; ST = State Transition; NM = Navigation Mapping; FE = Feature Execution; RS = Runtime Stability; RA = Rendering Availability.

Models	Semantic & Logical Consistency			Visual & Motion Fidelity			Structural & Temporal Coherence			Functional Executability			Avg.
	CC	WV	SI	MQ	TL	VH	LI	ST	NM	FE	RS	RA	
Closed-source Models													
Gemini-3.1 Pro	83.86	58.70	<u>70.54</u>	60.35	67.50	39.11	77.07	65.76	34.15	86.10	68.37	59.46	64.25
Gemini-3.1-Flash Lite	76.38	<u>72.83</u>	60.71	59.65	35.00	30.67	71.97	<u>74.46</u>	31.71	89.84	60.20	35.14	58.21
Gemini-3 Flash	81.50	66.30	66.07	35.09	<u>61.25</u>	40.00	41.40	66.30	14.63	83.42	64.29	64.86	57.09
Gemini-2.5 Flash	<u>85.43</u>	69.57	56.25	75.09	58.75	43.11	68.15	63.59	48.78	<u>90.37</u>	69.39	45.95	<u>64.54</u>
Open-weight models													
VideoLLaMA-3-7B	46.46	41.30	44.64	24.21	18.75	23.11	29.94	27.72	24.39	52.94	50.00	37.84	35.11
MiniCPM-V-4.5-8B	50.00	32.61	58.93	34.39	5.00	24.89	50.96	53.26	12.20	70.59	60.20	37.84	40.91
Qwen3-VL-8B Instruct	55.51	48.91	55.36	64.91	10.00	39.11	50.32	44.57	31.71	79.68	43.88	24.32	45.69
Qwen3-VL-8B Thinking	74.02	48.91	68.75	80.00	28.75	51.11	<u>78.98</u>	57.61	43.90	89.30	57.14	40.54	59.92
Qwen3-VL-30B-A3B-Instruct	61.02	43.48	68.75	75.79	13.75	31.11	71.97	58.15	26.83	86.10	55.10	35.14	52.27
Qwen3-VL-30B-A3B-Thinking	74.80	47.83	69.64	<u>84.56</u>	27.50	39.11	82.80	67.93	46.34	88.24	62.24	40.54	60.96
Qwen3.5-35B-A3B	83.46	66.30	71.43	69.47	47.50	36.89	63.69	75.54	<u>51.22</u>	83.96	63.27	54.05	63.90
GLM-4.6V-106B-A12B	63.78	52.17	58.93	63.86	18.75	36.89	66.88	58.70	31.71	77.01	<u>73.47</u>	43.24	53.78
Mimo-v2.5-310B-A15B	60.63	42.39	56.25	44.91	31.25	36.44	61.15	65.22	36.59	73.26	<u>52.04</u>	35.14	49.61
V2Lens (Ours)	87.01	78.26	65.18	85.96	58.75	<u>47.56</u>	72.61	<u>74.46</u>	58.54	93.58	76.53	<u>62.16</u>	71.72

Reviewer for independent verification. The Reviewer operates in a fresh context without access to the Evidence Judge’s internal reasoning or rationale. It independently adjudicates between the anonymized initial and challenge answers using the original video, webpage source code, diagnostic question, and candidate options. When necessary, the Reviewer may invoke the same evidence tools to conduct its own inspection. The second deterministic gate, the *Confirmation Gate*, converts the Reviewer’s judgment into the final prediction: the challenge answer replaces the initial answer only when it is independently endorsed by the Reviewer; otherwise, the initial answer is retained. Consequently, no single agent can unilaterally overwrite the initial diagnosis, and V2Lens revises its prediction only when an alternative survives both evidence screening and independent confirmation.

Experiments

Experimental Setup

Evaluated Models. We evaluate thirteen video-capable MLLMs, including four proprietary models and nine open-weight models. The proprietary models comprise Gemini-3.1-Pro, Gemini-3.1-Flash-Lite, Gemini-3-Flash, and Gemini-2.5-Flash (Comanici et al. 2025). The open-weight models include VideoLLaMA3 (Zhang et al. 2025), MiniCPM-V-4.5 (Yu et al. 2025), the Instruct and Thinking variants of Qwen3-VL at the 8B and 30B-A3B scales, and Qwen3.5-35B-A3B (Bai et al. 2025), GLM-4.6V (Hong et al. 2025), and MiMo-V2.5 (Li et al. 2025b).

Evaluation Protocol. We evaluate all models using a unified zero-shot, single-choice diagnostic protocol. Each instance provides an interaction video, the corresponding webpage source code, a diagnostic question, and a set of candidate failure modes. Unless otherwise specified, models jointly

analyze the video and source code under the *Video+HTML* setting. All models receive the same task instruction without in-context examples, and their free-form responses are deterministically mapped to the corresponding option labels.

Evaluation Metric. Each diagnostic instance belongs to one of the twelve evaluation subdimensions. For subdimension c containing N_c instances, let $y_{c,j}$ and $\hat{y}_{c,j}$ denote the ground-truth and prediction of its j -th instance, respectively. We compute the subdimension score and the overall score as

$$\text{Score}_c = \frac{1}{N_c} \sum_{j=1}^{N_c} \mathbf{1}[\hat{y}_{c,j} = y_{c,j}], \quad (1)$$

$$\text{Overall} = \frac{1}{12} \sum_{c=1}^{12} \text{Score}_c.$$

Here, $\mathbf{1}[\cdot]$ denotes the indicator function. We report the twelve subdimension scores and their unweighted macro-average, giving each subdimension equal weight regardless of its number of instances. All scores are reported as percentages.

Main Benchmark Results

Table 1 reports the diagnostic performance of video-capable MLLMs and V2Lens across the twelve evaluation subdimensions. V2Lens achieves the highest overall score of 71.72, outperforming the strongest standalone model, Gemini-2.5-Flash, by 7.18 percentage points. It also obtains the best scores in six subdimensions. Across standalone MLLMs, Feature Execution and Content Coherence are generally the strongest subdimensions, suggesting that directly observable functional failures and explicit semantic inconsistencies are comparatively easier to diagnose. Navigation Mapping and Visual Hierarchy remain more challenging, with the best standalone scores reaching only 51.22 and 51.11, respectively. Moreover, no standalone model consistently domi-

Table 2: Input-modality ablation for Gemini-2.5-Flash and Qwen3.5-35B-A3B on VideoVIBE.

Model	Input	Semantic & Logical Consistency			Visual & Motion Fidelity			Structural & Temporal Coherence			Functional Executability			Avg.
		CC	WV	SI	MQ	TL	VH	LI	ST	NM	FE	RS	RA	
Gemini-2.5-Flash	Video + HTML	85.43	69.57	<u>56.25</u>	75.09	58.75	43.11	68.15	63.59	48.78	90.37	69.39	45.95	64.54
	Video Only	<u>79.92</u>	50.00	64.29	<u>69.47</u>	45.00	48.44	69.43	66.85	56.10	<u>88.77</u>	77.55	54.05	<u>64.16</u>
	HTML Only	<u>79.92</u>	<u>58.70</u>	51.79	68.07	67.50	28.89	32.48	58.70	26.83	81.28	62.24	40.54	54.75
Qwen3.5-35B-A3B	Video + HTML	83.46	66.30	71.43	69.47	<u>47.50</u>	36.89	<u>63.69</u>	75.54	51.22	83.96	<u>63.27</u>	<u>54.05</u>	63.90
	Video Only	86.22	<u>63.04</u>	<u>70.54</u>	55.63	45.00	<u>29.78</u>	65.61	<u>63.04</u>	<u>36.59</u>	80.21	74.49	64.86	<u>61.25</u>
	HTML Only	75.59	55.43	69.64	<u>62.46</u>	62.50	27.56	21.66	55.98	12.20	89.30	47.96	37.84	51.51

Table 3: Component ablation of V2Lens on VideoVIBE.

Setting	Semantic & Logical Consistency			Visual & Motion Fidelity			Structural & Temporal Coherence			Functional Executability			Avg.
	CC	WV	SI	MQ	TL	VH	LI	ST	NM	FE	RS	RA	
Baseline (Gemini-2.5-Flash)	85.43	69.57	56.25	75.09	<u>58.75</u>	43.11	<u>68.15</u>	63.59	48.78	90.37	69.39	45.95	64.54
Full V2Lens	87.01	78.26	65.18	85.96	58.75	47.56	72.61	74.46	58.54	93.58	76.53	62.16	71.72
w/o Refinement Module	81.89	<u>73.91</u>	67.86	<u>84.56</u>	56.25	<u>44.89</u>	64.97	75.00	60.98	90.37	<u>73.47</u>	62.16	<u>69.69</u>
w/o HTML Verifier	<u>86.22</u>	72.83	59.82	77.54	61.25	43.56	59.87	68.48	51.22	<u>91.98</u>	69.39	54.05	66.35
w/o Evidence Tool	76.38	60.87	61.61	84.21	43.75	43.11	56.69	66.85	41.46	86.63	61.22	<u>56.76</u>	61.63

nates across all subdimensions. Qwen3.5-35B-A3B performs best on State Integrity, State Transition, and Navigation Mapping, while Qwen3-VL-30B-A3B-Thinking leads on Motion Quality and Layout Integrity. The smaller Qwen3-VL-8B-Thinking achieves the best Visual Hierarchy score, indicating that diagnostic accuracy does not exhibit a simple correspondence with model scale. Overall, current MLLMs remain limited on failures requiring precise reasoning over interaction targets, evolving application states, dynamic feedback, and fine-grained visual relationships.

Ablation Studies

Input Modalities. Table 2 examines the contributions of interaction videos and webpage HTML source code. Removing the video modality (*HTML only*) reduces the overall scores of Gemini-2.5-Flash and Qwen3.5-35B-A3B from 64.54 to 54.75 and from 63.90 to 51.51, respectively. The largest degradations occur in Layout Integrity and Navigation Mapping, showing that source code alone cannot reliably determine how user actions affect interface responses and application states. By contrast, the *Video-only* setting retains most of the diagnostic performance, reaching 64.16 for Gemini-2.5-Flash and 61.25 for Qwen3.5-35B-A3B. This result confirms that interaction videos provide the primary evidence for failure diagnosis. Nevertheless, source code offers complementary, model-dependent benefits. For Qwen3.5-35B-A3B, adding source code substantially improves Motion Quality, State Transition, and Navigation Mapping. These results support the V2Lens design: videos capture observable failure behavior, while source code provides complementary implementation evidence to distinguish confusable diagnoses.

V2Lens Components. Table 3 evaluates the contribution of each V2Lens component using Gemini-2.5-Flash as the base evaluator. The complete system improves the overall score from 64.54 to 71.72. The largest gains occur in Rendering Availability (+16.21), Motion Quality and State Transition

(+10.87 each), and Navigation Mapping (+9.76), indicating that evidence-grounded verification is particularly effective for failures involving dynamic behavior, evolving states, and implementation-dependent effects. Removing the *Refinement Module* causes the Evidence Judge’s challenge answer to be accepted directly and reduces the score to 69.69, demonstrating the value of independently verifying proposed revisions. Removing the *HTML Verifier* produces a larger decline to 66.35, with pronounced losses in Layout Integrity, Motion Quality, Rendering Availability, Navigation Mapping, and Runtime Stability. This result highlights the benefit of a dedicated source-code verification stage beyond ad hoc inspection by the *Evidence Judge*. The largest degradation occurs when the evidence tools are removed: performance falls to 61.63, which is 10.09 points below the complete system and even below the standalone baseline. Thus, the gains of V2Lens arise not merely from multi-agent deliberation, but from structured, tool-grounded evidence collection followed by conservative refinement. Additional qualitative analyses, including visualized examples of successful and failed diagnoses across different MLLMs, are provided in Appendix.

Conclusion

We introduced **VideoVIBE**, a video-grounded benchmark for fine-grained failure diagnosis in one-shot-generated interactive web applications. Grounded in human-operated recordings, it evaluates both rendered presentation and application behavior through a hierarchical failure taxonomy. Experiments across closed-source and open-weight Video MLLMs reveal substantial room for improvement and no consistently dominant standalone model. We further proposed **V2Lens**, a training-free multi-agent system that integrates video and complementary source-code evidence, improving its base evaluator by 7.18 points. VideoVIBE advances generated-webpage evaluation toward more evidence-grounded and diagnostically informative assessment.

References

- Awal, R.; Massoud, M.; Feizi, A.; Li, Z.; Wang, S.; Pal, C.; Agrawal, A.; Vazquez, D.; Reddy, S.; Rodriguez, J. A.; et al. 2025. Webmmu: A benchmark for multimodal multilingual website understanding and code generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 25129–25156.
- Bai, S.; Cai, Y.; Chen, R.; Chen, K.; Chen, X.; Cheng, Z.; Deng, L.; Ding, W.; Gao, C.; Ge, C.; et al. 2025. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.
- Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; Blistein, M.; Ram, O.; Zhang, D.; Rosen, E.; et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Dai, Y.; Lai, Y.; Huang, M.; Guo, H.; Li, D.; Peng, H.; Li, H.; Zhao, Y.; Lyu, H.; Ge, Z.; et al. 2026. Webvr: Benchmarking multimodal llms for webpage recreation from videos via human-aligned visual rubrics. *arXiv preprint arXiv:2603.13391*.
- He, Z.; Hong, W.; Yang, Z.; Pan, Z.; Liu, M.; Gu, X.; and Tang, J. 2026. Vision2web: A hierarchical benchmark for visual website development with agent verification. *arXiv preprint arXiv:2603.26648*.
- Hong, S.; Liu, Z.; Li, T.; Tao, W.; Wu, Y.; and Wu, C. 2026. DiagEval: Trajectory-Conditioned Diagnosis for Reliable Software Evaluation with GUI Agents. *arXiv preprint arXiv:2605.17439*.
- Hong, W.; Yu, W.; Gu, X.; Wang, G.; Gan, G.; Tang, H.; Cheng, J.; Qi, J.; Ji, J.; Pan, L.; et al. 2025. Glm-4.5 v and glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006*.
- Jang, L.; Li, Y.; Zhao, D.; Ding, C.; Lin, J.; Liang, P. P.; Bonatti, R.; and Koishida, K. 2024. Videowebarena: Evaluating long context multimodal agents with video understanding web tasks. *arXiv preprint arXiv:2410.19100*.
- Jiang, J.; Cai, C.; Park, C.; Shen, J.; Kim, S.; Li, J.; and Wang, Y. 2026. WebGen-R1: Incentivizing Large Language Models to Generate Functional and Aesthetic Websites with Reinforcement Learning. *arXiv preprint arXiv:2604.20398*.
- Li, C.; Zheng, Y.; Huang, X.; Fang, T.; Xu, J.; Chen, L.; Song, Y.; and Hu, H. 2025a. Webdevjudge: Evaluating (m) llms as critiques for web development quality. *arXiv preprint arXiv:2510.18560*.
- Li, J.; Chen, J.; Qu, Y.; Xu, S.; Lin, Z.; Zhu, J.; Xu, B.; Tan, W.; Fu, P.; Ju, J.; et al. 2025b. Xiaomi MiMo-VL-Miloco Technical Report. *arXiv preprint arXiv:2512.17436*.
- Li, Y.; Zeng, Z.; Yang, Y.; Yang, Y.; Liao, N.; Guo, W.; Qiu, L.; Cheng, M.; Dai, Q.; Wang, Z.; et al. 2026. MM-WebAgent: A Hierarchical Multimodal Web Agent for Webpage Generation. *arXiv preprint arXiv:2604.15309*.
- Lin, K. Q.; Hu, S.; Li, L.; Yang, Z.; Wang, L.; Torr, P.; and Shou, M. Z. 2025a. Computer-Use Agents as Judges for Generative User Interface. *arXiv preprint arXiv:2511.15567*.
- Lin, K. Q.; Li, L.; Gao, D.; Wu, Q.; Yan, M.; Yang, Z.; Wang, L.; and Shou, M. Z. 2024. VideoGUI: A Benchmark for GUI Automation from Instructional Videos. In *NeurIPS*.
- Lin, Z.; Zhou, Z.; Zhao, Z.; Wan, T.; Ma, Y.; Gao, J.; and Li, X. 2025b. Webuibench: a comprehensive benchmark for evaluating multimodal large language models in webui-to-code. In *Findings of the Association for Computational Linguistics: ACL 2025*, 15780–15797.
- Liu, C.; Fu, Y.; Yang, W.; Zhang, Y.; and Xie, T. 2026. WebCoderBench: Benchmarking Web Application Generation with Comprehensive and Interpretable Evaluation Metrics. *arXiv preprint arXiv:2601.02430*.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*.
- Lu, Z.; Yang, Y.; Ren, H.; Hou, H.; Xiao, H.; Wang, K.; Shi, W.; Zhou, A.; Zhan, M.; and Li, H. 2026. Webgenbench: Evaluating llms on generating interactive and functional websites from scratch. *Advances in Neural Information Processing Systems*, 38.
- Wang, X.; Li, B.; Song, Y.; Xu, F. F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; et al. 2025. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, 65882–65919.
- Xiao, J.; Wang, M.; Lam, M. H.; Wan, Y.; Liu, J.; Huo, Y.; and Lyu, M. R. 2025. Designbench: A comprehensive benchmark for mllm-based front-end code generation. *arXiv preprint arXiv:2506.06251*.
- Xu, M.; Yang, Z.; Wang, Y.; Wang, Y.; Liu, X.; Dou, Z.; Hong, W.; Gu, X.; Xu, B.; and Tang, J. 2026. Video2Code: Generating Interactive Webpages from UI Videos via Action-Aware Revisit. *arXiv preprint arXiv:2606.20711*.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K. R.; and Press, O. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Yu, T.; Wang, Z.; Wang, C.; Huang, F.; Ma, W.; He, Z.; Cai, T.; Chen, W.; Huang, Y.; Zhao, Y.; et al. 2025. Minicpm-v 4.5: Cooking efficient mllms via architecture, data, and training recipe. *arXiv preprint arXiv:2509.18154*.
- Yun, S.; Lin, H.; Thushara, R.; Bhat, M. Q.; Wang, Y.; Jiang, Z.; Deng, M.; Wang, J.; Tao, T.; Li, J.; et al. 2024. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms. *Advances in neural information processing systems*, 37: 112134–112157.
- Zhang, B.; Li, K.; Cheng, Z.; Hu, Z.; Yuan, Y.; Chen, G.; Leng, S.; Jiang, Y.; Zhang, H.; Li, X.; et al. 2025. Video-ollama 3: Frontier multimodal foundation models for image and video understanding. *arXiv preprint arXiv:2501.13106*.
- Zhao, Y.; Yang, Z.; Chen, M.; Xu, M.; Gong, S.; Liu, X.; Gong, J.; and Tang, J. 2026. LongWebBench: Evaluating Structural and Functional Webpage Generation in Long-Horizon Settings. *arXiv preprint arXiv:2606.17727*.

A Prompt Design for Benchmark Construction and Evaluation

To ensure consistency in web application construction and rigor in model evaluation, we design a series of structured system prompts that specify the roles, inputs, task constraints, and output formats of the models at different stages.

A.1 Web Application Construction Prompts

We first use the **Interactive Web Application Specification Generation Prompt** to generate diverse webpage-generation instructions, as shown in Fig. 12. The resulting application descriptions and visual styles are then provided to the **Web Application Generation Prompt**. This prompt further instructs the webpage-generation model to produce a complete single-page interactive application containing HTML, CSS, and JavaScript, while implementing the specified interaction workflow, state logic, visual design, and responsive layout, as shown in Fig. 13. Using these prompts, we generated a total of 700 interactive HTML applications for benchmark construction.

A.2 Failure Candidate Discovery Prompt

The **Failure Candidate Discovery Prompt** is used to identify potential webpage failures from complete interaction recordings. It instructs the model to locate time intervals containing possible anomalies based on observable visual behavior and to provide concise descriptions of the corresponding events, as shown in Fig. 14.

The discovered failure candidates are subsequently inspected by annotators using the complete interaction recordings and corresponding webpage source code.

A.3 Video MLLM Evaluation Prompts

During benchmark evaluation, each evaluated video multi-modal large language model (Video MLLM) is prompted to act as a professional QA auditor for generated interactive web applications. For each diagnostic question, the model is provided with the available modality input and a set of candidate failure diagnoses. It is required to examine the available evidence and select the candidate diagnosis that is best supported by the observed interaction behavior or implementation-level evidence.

We design three evaluation prompt templates corresponding to different input modalities:

1. **Video–HTML Evaluation** (Fig. 15): the evaluated MLLM diagnoses the failure using both the interaction recording and the webpage source code.
2. **Video-Only Evaluation** (Fig. 16): the evaluated MLLM diagnoses the failure using only the interaction recording.
3. **HTML-Only Evaluation** (Fig. 17): the evaluated MLLM diagnoses the failure using only the webpage source code.

All three templates use the same diagnostic questions, candidate options, and structured output format. Their evidence instructions are adapted to the available input modality, enabling a controlled and fair comparison across input settings.

A.4 V2Lens Multi-Agent Prompts

We further design a unified **Multi-Agent Prompt Suite** for V2Lens, containing separate system prompts for four specialized roles: **Video Analyst**, **HTML Verifier**, **Evidence Judge**, and **Reviewer**.

The Video Analyst forms the initial diagnosis from the complete interaction recording and evaluates the observable evidence for each candidate. The HTML Verifier inspects the webpage source code and determines whether the implementation-level evidence supports, contradicts, or remains inconclusive for each candidate diagnosis. The Evidence Judge integrates the initial diagnosis, video observations, and source-code findings, and proposes a challenge answer when an alternative receives stronger evidential support. The Reviewer independently compares the anonymized initial and challenge answers in an isolated context. Each prompt specifies the corresponding agent’s task boundaries, evidence requirements, and structured output format, as shown in Figs. 18–21.

B Detailed Descriptions of Failure Modes

Tables 5–8 present the fine-grained failure taxonomy of VideoVIBE, comprising 30 failure modes organized under four top-level dimensions: *Semantic & Logical Consistency*, *Visual & Motion Fidelity*, *Structural & Temporal Coherence*, and *Functional Executability*.

Together, these definitions establish the evaluation scope of the benchmark and provide a unified basis for failure annotation, diagnostic QA construction, and model evaluation.

C Annotation Quality and Reliability

To ensure annotation consistency and reliability, we adopt a multi-stage quality-control process that combines automated candidate screening, multi-annotator review, and expert adjudication. Our annotation team consists of 17 trained annotators with master’s or doctoral backgrounds. All annotation and verification procedures are conducted through a custom-built platform that jointly presents the interaction recordings, failure taxonomy, and candidate annotations, as shown in Fig. 11.

Quality control is applied throughout both failure annotation and diagnostic QA construction. During failure annotation, automated video analysis first uses the Failure Candidate Discovery Prompt to identify potential anomalies and their temporal intervals in the complete interaction recordings. Annotators then manually verify each candidate by jointly inspecting the full recording and the corresponding webpage source code. A candidate is retained only when its manifestation is observable, reproducible, and clearly aligned with a fine-grained taxonomy definition. Automated tools are used only for candidate discovery, annotation organization, and consistency checking; all final failure labels and supporting evidence are manually verified.

During diagnostic QA construction, each video–top-level-dimension pair is treated as the unit of representative selection, and at most one single-choice question is retained for each pair. Multiple annotators then independently review the question wording, candidate options, ground-truth diagnosis, and their consistency with the verified evidence under shared annotation

Method	Mean Time/Q	Model Stages/Q	Tool Calls/Q	Avg. Acc.
Gemini-2.5-Flash	29.65 s	1.00	0.00	64.54%
V2Lens	100.94 s	3.17	9.61	71.72%

Table 4: Per-question inference cost and diagnostic performance.

guidelines. Question–answer pairs with consistent judgments pass the verification stage, whereas disagreements are resolved through discussion and further revision. Unresolved cases are submitted for expert adjudication. Only manually verified failure annotations are included in the final benchmark, and all retained diagnostic questions must pass representative selection and multi-stage verification.

D Additional Analysis of V2Lens

Inference Cost and Diagnostic Gain. Table 4 compares the per-question inference cost and diagnostic performance of V2Lens with its backbone model, Gemini-2.5-Flash. The single-stage baseline requires 29.65 seconds per question, whereas V2Lens takes 100.94 seconds on average. V2Lens executes an average of 3.17 model stages and 9.61 tool calls per question, reflecting the additional cost introduced by multi-stage evidence verification. Despite this overhead, V2Lens improves average accuracy from 64.54% to 71.72%, yielding an absolute gain of 7.18 percentage points. These results demonstrate a clear trade-off: V2Lens incurs a higher inference cost in exchange for a substantial improvement in diagnostic reliability.

E Data Distribution Analysis

Fig. 10 shows the distribution of the 1,752 diagnostic questions across the failure taxonomy. Visual & Motion Fidelity accounts for the largest proportion (33.7%), driven primarily by Motion Quality (16.3%) and Visual Hierarchy (12.8%). Semantic & Logical Consistency follows at 26.1%, with Content Coherence contributing 14.5%. Structural & Temporal Coherence represents 21.8%, with substantial coverage of State Transition (10.5%) and Layout Integrity (9.0%). Functional Executability constitutes the remaining 18.4%, led by Feature Execution (10.7%). Overall, the distribution emphasizes visually and temporally observable failures while maintaining broad coverage across the semantic, structural, and functional aspects of interactive web applications.

F Qualitative Case Studies

We provide four qualitative cases to illustrate how VideoVIBE distinguishes model capabilities across different levels of granularity.

Semantic & Logical Consistency. Fig. 5 presents a failure case involving a logical constraint in a generated personal-finance webpage. Although each bill should be assigned to only one category, the interface allows the same bill to be allocated to two budget categories, causing its amount to be counted twice. The system provides no validation warning

and still permits the user to proceed to the next page after the inconsistency occurs. Gemini-2.5-Flash and Qwen3.5-35B correctly classify this failure as a validation-rule violation (Option B), demonstrating their ability to identify subtle logical constraints. In contrast, GLM-4.6V fails to recognize the underlying semantic and logical anomaly and instead selects the language-violation category. This case highlights the considerable difficulty of diagnosing fine-grained business-logic validation failures in generated web applications.

Visual & Motion Fidelity. Fig. 6 presents a failure case involving visual fidelity. The watch preview is rendered using only flat two-dimensional graphics, without realistic material textures or refined luxury-oriented details. Its visual elements appear simplistic and coarse, resulting in an unfinished, low-fidelity interface. Gemini-2.5-Flash, GLM-4.6V, and Qwen3.5-35B all correctly classify this failure as a low-visual-fidelity defect (Option C), demonstrating their ability to identify insufficient visual refinement and distinguish between different levels of visual fidelity. None of the three models produces an incorrect prediction, and all accurately identify the lack of detailed rendering and premium visual quality as the central issue. This case suggests that models can reliably recognize visually salient low-fidelity defects, although assessing more subtle differences in rendering quality remains challenging in generated web applications.

Structural & Temporal Coherence. Fig. 7 presents a failure case involving cross-region interface-state synchronization. After the user drags an asset from the index panel on the left into the central display area, the display area is correctly updated to show the transferred asset. However, the corresponding item in the asset index should simultaneously become unavailable for further selection, whereas it remains active and interactive. As a result, conflicting interface states coexist across different regions of the page. Qwen3.5-35B correctly classifies this failure as a cross-region state mismatch (Option A), demonstrating its ability to identify synchronization inconsistencies across multiple interface regions. In contrast, Gemini-2.5-Flash and GLM-4.6V both fail to recognize the synchronization failure and instead select categories related to state transitions. This case highlights the considerable difficulty of diagnosing subtle cross-region state-synchronization failures in generated web applications.

Functional Executability. Fig. 8 presents a failure case involving the usability of an interactive control. The user enters a transfer amount of 33,332 in a cross-border transfer form, but the subsequent confirmation page displays a fixed amount of \$500,000. The input control therefore fails to propagate the user-provided value correctly, preventing the interaction from producing the expected task outcome. Gemini-2.5-Flash and Qwen3.5-35B correctly classify this failure as an interactive-control malfunction (Option C), demonstrating their ability to identify anomalies in interaction execution logic. In contrast, GLM-4.6V incorrectly determines that no functional execution failure is present and fails to capture the underlying data-propagation error. This case highlights the considerable difficulty of diagnosing subtle interaction-execution failures in generated web applications.

G Qualitative Case Study of V2Lens Multi-Agent Reasoning

Fig. 9 illustrates how V2Lens corrects a misdiagnosis caused by a salient surface-level cue under the Semantic & Logical Consistency dimension. Multiple baseline models selected Option A, and the Video Analyst produced the same initial answer after observing an incompatibility notification in the video, incorrectly interpreting this normally triggered message, which successfully prevented an invalid selection, as a compatibility-validation failure. The HTML Verifier then used the *HTML Inspection* tool to progressively retrieve relevant source-code fragments using keywords supplied by the Video Analyst, and successfully located anomalous logic in the engraving-field processing: the code uses `value === 'engraving'` to determine the field type instead of checking the corresponding key. This implementation evidence suggested that the engraving input might not be correctly retrieved and propagated to the final configuration summary. The Evidence Judge further integrated this code-level clue with the interaction video and used the *Frame Extraction* tool to inspect the relevant steps. It found that the user entered the engraving text “33333” during customization, whereas the final summary displayed the field as “undefined,” indicating that the input was not correctly preserved and propagated across successive interaction steps. The Judge therefore proposed the state-continuity failure described by Option B as the challenger answer. After passing the Evidence Gate, the challenger was submitted to an independent Reviewer, which further confirmed that both the observed video behavior and the implementation logic supported Option B, leading to the final answer correction. This case represents a highly fine-grained interaction failure: although the overall workflow remains executable, a specific user input is not correctly propagated or presented. Identifying this underlying state-continuity failure requires jointly examining temporal video evidence, key frames, and retrieved source-code logic.

Subdimension	Failure Mode	Description
Feature Execution	Non-functional interactive control	UI controls such as buttons, input fields, search, filters, playback, or submit actions appear operable, but after activation they produce no meaningful feedback, state change, or task outcome.
Runtime Stability	Interaction flow stuck	After a user action, the page remains stuck in a loading, processing, current-step, or loop state and cannot advance to the next step.
	Excessive interaction delay	The system may eventually respond, but the wait time is clearly excessive and there is no progress indication, skip option, or user control mechanism.
	Missing failure feedback	After invalid input, a failed operation, or a failed submission, the system provides no error message, cause, or recovery path.
	Progressive performance degradation	Page responsiveness progressively deteriorates during use; repeated clicks or drags make responses slower, scrolling becomes sluggish, or animations drop frames.
Rendering Availability	Broken media resource	Resources that should be displayed or played, such as images, video, or audio, fail to load correctly.
	Page content rendering failure	Primary or partial content does not display correctly during initial load, scrolling, switching, navigation, or state updates, appearing blank, missing, misplaced, or incompletely rendered.

Table 5: Functional Executability Main Dimension

Subdimension	Failure Mode	Description
Layout Integrity	Redundant interface element	The page contains irrelevant, duplicated, or unexplainable containers, windows, overlays, modules, or components.
	Occlusion and layer conflict	Page elements, overlays, modals, masks, or background content occlude, overlap, or stack incorrectly, making content unreadable, controls unusable, or the active interaction area ambiguous.
	Responsive adaptation failure	When the viewport is resized, orientation changes, or the page is previewed on mobile, the layout fails to adapt and shows misalignment, occlusion, or broken structure.
State Transition	Discontinuous state transition	The page state changes abruptly, or a process skips required intermediate steps and jumps directly to a later state, making the flow relationship difficult to understand.
	Cross-region state mismatch	After a user action, only part of the page updates to the new state while related areas remain in the previous state, causing old and new states or inconsistent information to coexist in one interface.
Navigation Mapping	Broken link	A link or navigation entry is present, but clicking it fails to open the target, leads to an empty link, returns a 404, or has no destination page.
	Wrong interaction target	Clicking a control triggers the wrong page, modal, object, or function.

Table 6: Structural & Temporal Coherence Main Dimension

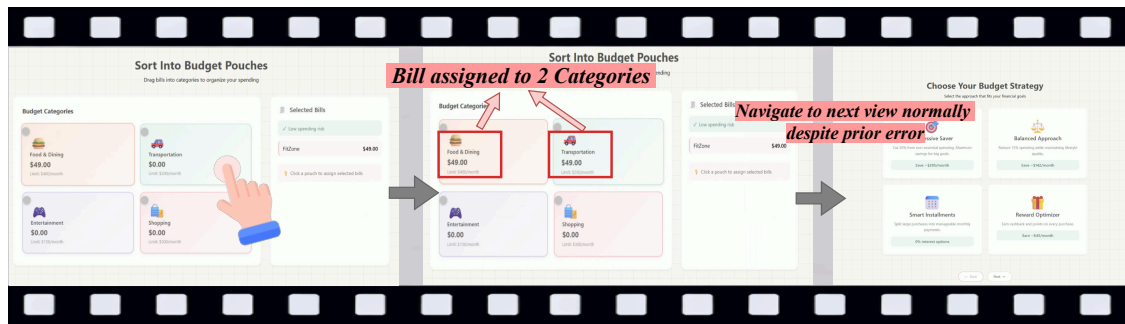
Subdimension	Failure Mode	Description
--------------	--------------	-------------

Motion Quality	Motion feedback failure	Expected motion is missing, or motion direction, speed, trigger timing, or rhythm conflicts with the state change, making the transition hard to understand.
Theme & Legibility	Theme consistency failure	Backgrounds, theme modes, or the overall visual atmosphere are inconsistent across areas or states, such as unexplained background changes, mixed light and dark modes, or discontinuous theme styling.
	Color scheme conflict	The page uses colors that are overly bright, oversaturated, strongly conflicting, or too many similar high-saturation hues, causing visual fatigue or a chaotic impression.
	Text color contrast failure	Text color conflicts with the background or overlay, making content difficult or impossible to read.
Visual Hierarchy	Visual hierarchy failure	The page fails to emphasize core information through size, contrast, position, whitespace, or grouping, making reading order or action priorities unclear.
	Visual style inconsistency	Fonts, icons, colors, corner radii, shadows, materials, spacing, or component styles are inconsistent.
	Low visual fidelity	Details such as graphics, images, materials, edges, shadows, or typography are rough, making the interface look like an unfinished low-fidelity page.
	Generic template-like layout	The page relies heavily on generic cards, repeated modules, or common templates and lacks design customization relevant to the target website, industry, or task.

Table 7: Visual & Motion Fidelity Main Dimension

Subdimension	Failure Mode	Description
Content Coherence	Website Purpose Mismatch	Page features, flows, recommendations, content, or results do not match the website's business purpose; for example, an education site presents a shopping flow, or a medical appointment page shows travel packages.
	Asset-content mismatch	Images, products, services, copy, headings, or video content are inconsistent with the page topic, task objective, or description.
	Internal content inconsistency	Values, labels, headings, charts, lists, or descriptions on the same page or across consecutive states contradict one another.
Workflow Validity	Validation rule failure	The system fails to correctly enforce validation rules such as required fields, permissions, login, authorization, quantity, amount, date, or format constraints.
	Language requirement violation	Application specification requires a specific interface language, but the page contains content in another, non-target language.
State Integrity	Non-interactive scripted demo state	The page appears interactive, but user input is not retained or used; subsequent changes follow preset scripts or animations.
	State carryover failure	User selections, inputs, progress, or results are not correctly carried into later steps, causing lost context, default values, or inconsistent outcomes.
	Unreplaced placeholder exposure	The page directly exposes unprocessed template variables, placeholder text, debug values, or null-value strings.

Table 8: Semantic & Logical Consistency Main Dimension



Questions & Answers

You are evaluating a generated website used for: "Support younger users with an online system centered on spending timelines, budget pouches, and installment comparison workflows and related task coordination". Considering only **Semantic & Logical Consistency**, select the single failure mode that best matches the observable behavior in the video. Include a brief video-based rationale.

- A. Language requirement violation : The prompt, source video, or task explicitly requires a specific interface language, but the page contains content in another, non-target language.
- B. Validation rule failure : The system fails to correctly enforce validation rules such as required fields, permissions, login, authorization, quantity, amount, date, or format constraints. **Correct Answer**
- C. Asset-content mismatch : Images, products, services, copy, headings, or video content are inconsistent with the page topic, task objective, or description.



Gemini-2.5
-Flash

Choice: B. The system fails to enforce a fundamental validation rule for budgeting: a single bill should logically be assigned to only one category (or handled with a specific split mechanism, which is not present). As seen when the user assigns the 'FitZone' bill to 'Food & Dining' and then to 'Transportation', the same bill's amount is added to both categories without any warning or logical correction, leading to double-counting and an inconsistent budget.



GLM-4.6V

Choice: A. The video shows a budgeting application with consistent English language throughout, no validation rule failures, and content that matches the budgeting theme. However, since I must select one option, and considering the semantic and logic consistency of the application, I'll select the option that represents the absence of semantic and logic consistency issues.



Qwen3.5
-35B

Choice: B. During the budget categorization process, the system exhibits incorrect logic. Specifically, returning to Step 2 results in the 'Food & Dining' budget total displaying \$49.00 and the subsequent clicking of pouches duplicates the \$49.00 amount across different categories inconsistently. This violates the logical expectation of how bill categorization and budget sums should work.

Figure 5: Case study of Semantic & Logical Consistency



Questions & Answers

This video shows a user operating a vibe-coded website for: "Create an online workspace where luxury shoppers can coordinate related tasks while managing watch components, preview scenes, and order workflows". Focusing only on **Visual & Motion Fidelity**, which failure mode is most directly supported by the video evidence? Choose one option and provide the observed evidence.

- A. Visual hierarchy failure : The page fails to emphasize core information through size, contrast, position, whitespace, or grouping, making reading order or action priorities unclear.
- B. Visual style inconsistency : Fonts, icons, colors, corner radii, shadows, materials, spacing, or component styles are inconsistent.
- C. Low visual fidelity : Details such as graphics, images, materials, edges, shadows, or typography are rough, making the interface look like an unfinished low-fidelity page. ✓ **Correct Answer**



Gemini-2.5-Flash

Choice: C. The watch preview in the configurator exhibits low visual fidelity. The rendering of the watch case, dial, and hands uses basic gradients and simple shapes, lacking realistic textures, intricate details, and sophisticated lighting effects typically expected for a luxury product configurator. The hour markers, for example, are simple rectangles, which contributes to an unfinished, low-fidelity appearance. ✓



GLM-4.6V

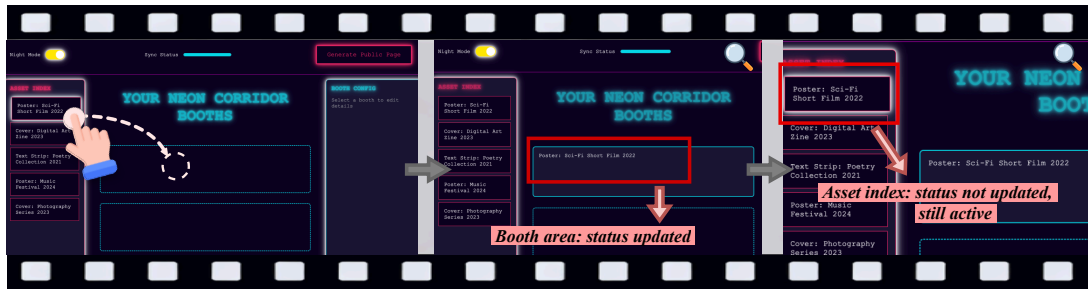
Choice: C. The watch preview in the configurator workspace shows basic watch elements with minimal detail - the hour markers are simple lines, the hands are basic shapes, and the overall visual representation lacks the refinement and detail expected in a luxury watch configuration interface, indicating low visual fidelity that makes the interface appear unfinished. ✓



Qwen3.5-35B

Choice: C. The visual assets and rendering exhibit signs of low visual fidelity relative to the luxury theme. The watch preview is constructed from basic CSS circles and gradients rather than realistic materials or textures. Furthermore, the complication icons utilize generic system emojis instead of custom branded vector graphics, contributing to an appearance that resembles an unfinished or low-fidelity prototype rather than a high-end bespoke workshop interface. ✓

Figure 6: Case study of Visual & Motion Fidelity



Questions & Answers

The video records a user workflow on a vibe-coded website for: "Design a task-focused site that brings neon-corridor booths, project assets, and portfolio-gallery workflows into one workspace for creators to coordinate related tasks". Under **Structural & Temporal Coherence**, which one listed failure mode is verified by the video? Choose one answer and cite the relevant evidence.

- A. Cross-region state mismatch : After a user action, only part of the page updates to the new state while related areas remain in the previous state, causing old and new states or inconsistent information to coexist in one interface. **Correct Answer**
- B. Discontinuous state transition : The page state changes abruptly, or a process skips required intermediate steps and jumps directly to a later state, making the flow relationship difficult to understand.
- C. Wrong interaction target : Clicking a control triggers the wrong page, modal, object, or function.



Gemini-2.5
-Flash

Choice: B. Throughout the video, the transitions between different major screens are abrupt, appearing instantaneously without any smooth animations or intermediate steps. Examples include the transition from the welcome screen to 'Organize Your Work', from 'Select Your Workflow' to the main workspace, and from the workspace to the 'Exhibition Live!' result screen. Similarly, the appearance of the booth configuration form upon selecting a booth is immediate.



GLM-4.6V

Choice: B. The video demonstrates a smooth workflow where user actions (dragging assets, clicking buttons) result in expected state changes. The sync indicator progresses, and the final result screen appears after the 'Generate Public Page' button is clicked. There are no abrupt state transitions, partial updates, or incorrect interactions shown.

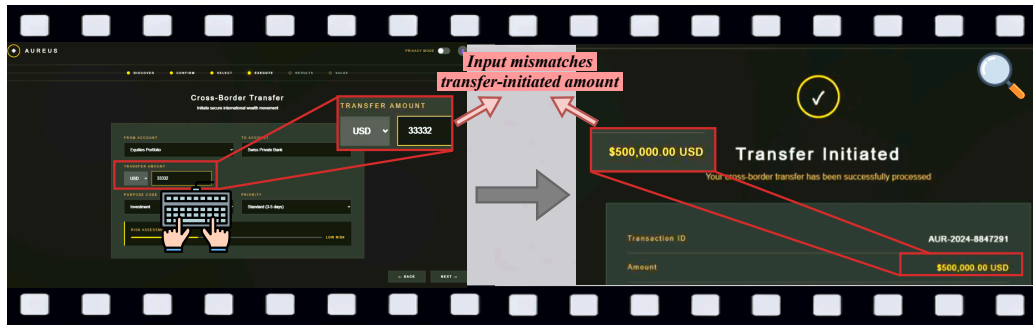


Qwen3.5
-35B

Choice: A. During the task of managing assets, the user drags items from the 'Asset Index' on the left to the 'Booths' in the center. While the Center region updates to show filled booths (new state), the Left region ('Asset Index') does not update its state to reflect that the items have been moved or used (they remain available and active). This results in a cross-region state mismatch where related areas hold contradictory states simultaneously.



Figure 7: Case study of Structural & Temporal Coherence



Questions & Answers

This video shows a user operating a vibe-coded website for: "Design a task-focused site that brings wallet views, masked asset panels, and transfer workflows into one workspace for high-net-worth users to coordinate related tasks". Focusing only on **Functional Executability**, which failure mode is most directly supported by the video evidence? Choose one option and provide the observed evidence.

- A. Broken media resource : Resources that should be displayed or played, such as images, video, or audio, fail to load correctly.
 - B. Page content rendering failure : Primary or partial content does not display correctly during initial load, scrolling, switching, navigation, or state updates, appearing blank, missing, misplaced, or incompletely rendered.
 - C. Non-functional interactive control : UI controls such as buttons, input fields, search, filters, playback, or submit actions appear operable, but after activation they produce no meaningful feedback, state change, or task outcome.
- ✓ **Correct Answer**



Gemini-2.5-Flash

Choice: C. The transfer amount input field does not correctly carry the user's input to the next stage of the workflow. The user types '33332' into the transfer amount field, but the subsequent 'Transfer Initiated' screen displays a different amount, '\$500,000.00 USD', indicating that the input control failed to produce the intended task outcome.

✓



GLM-4.6V

Choice: A. The video shows a complete workflow where all interactive controls function as expected. The user can select sources, configure privacy settings, select assets, fill out transfer forms, and see results. All buttons enable/disable appropriately and produce the expected state changes. No functional execution failures are observable in the video evidence.

✗



Qwen3.5-35B

Choice: C. The user enters specific values and makes selections in the 'Cross-Border Transfer' form, but the subsequent 'Results' screen displays different data (Amount \$500,000 vs Input 33332). This indicates the interactive controls failed to correctly process and transmit the user's input, fitting the definition of a non-functional interactive control where the task outcome does not match the activation.

✓

Figure 8: Case study of Functional Executability

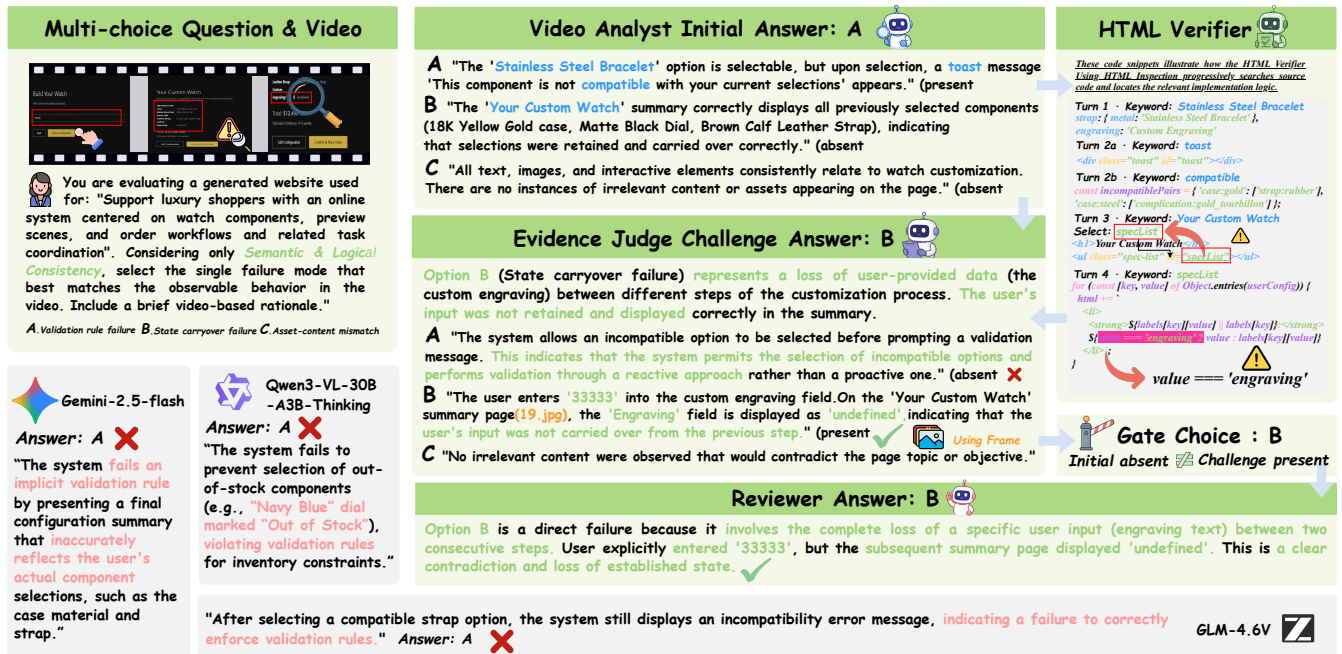


Figure 9: Qualitative Case Study of V2Lens under the Semantic & Logical Consistency Dimension

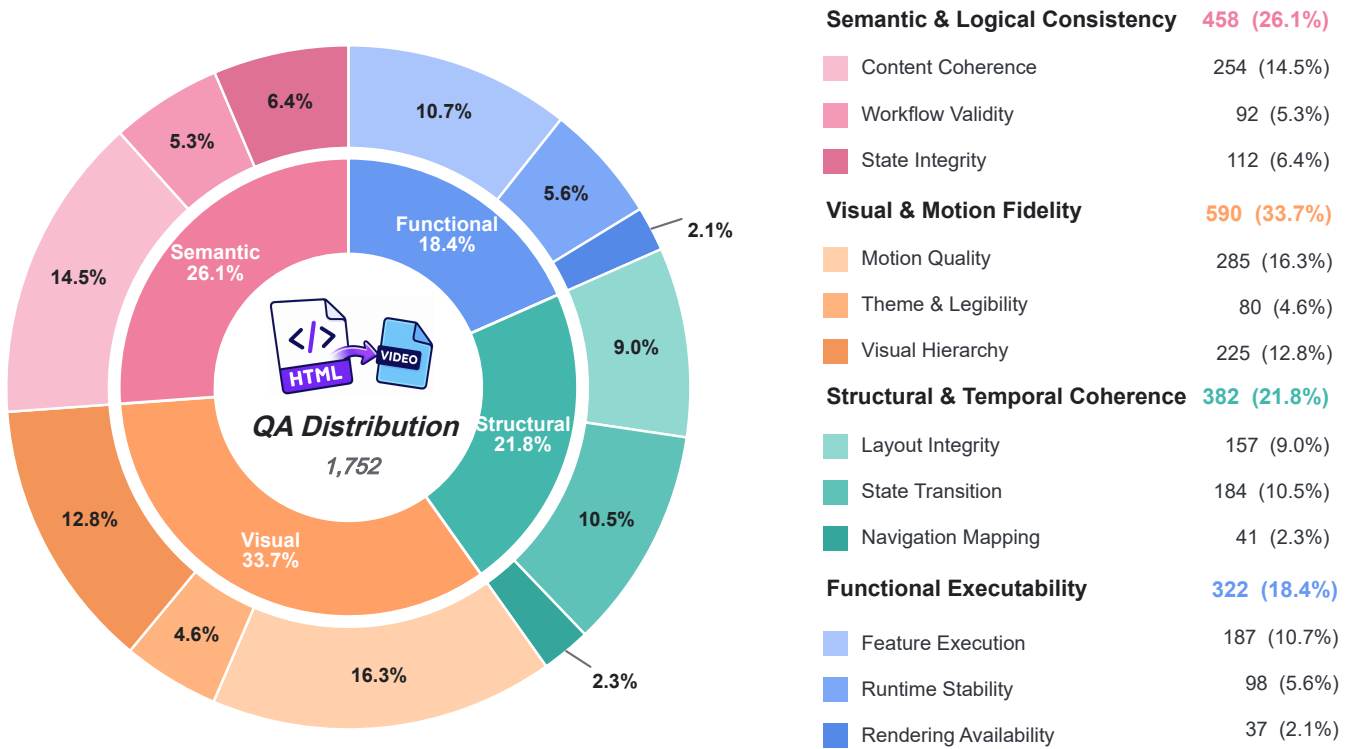


Figure 10: Distribution of diagnostic questions across the VideoVIBE

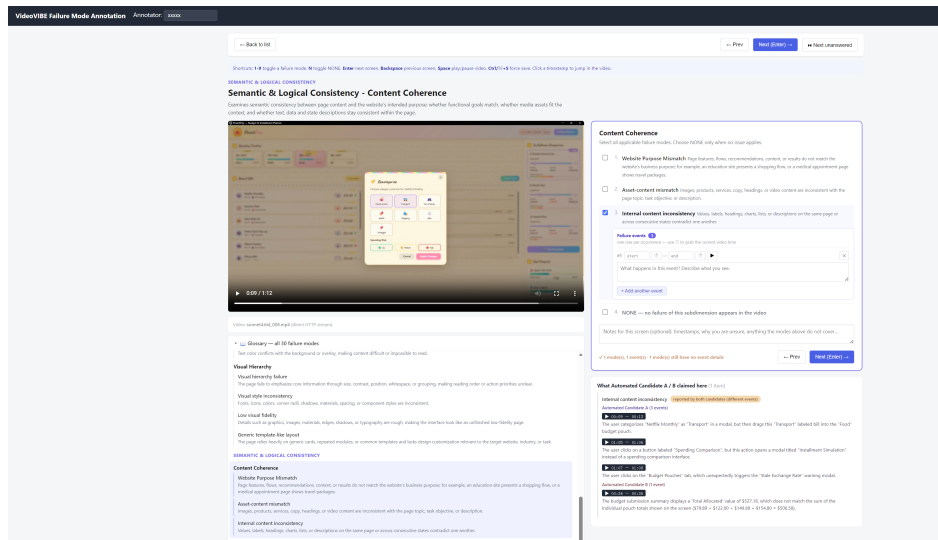


Figure 11: Customized Annotation Platform

System Instruction

Interactive Web Application Specification Generation Prompt

You are a senior interactive web application architect and interaction designer. Your task is to generate {num_samples} diverse specifications for self-contained, visually distinctive, and stateful web applications. Each specification must describe a realistic user goal, a complete interaction workflow, observable state changes, a recovery path, and a clear final outcome. Describe intended application behavior only.

1. Domain and Application Coverage

Distribute the applications across Technology; Finance & Payments; Medical & Health; Education & Training; Retail & E-commerce; Entertainment & Games; Creative & Media; and Public Service & Government.

Each application should primarily follow one archetype: **Functional**, **Creative**, **Tool**, or **Record-and-Tracking**.

Across the generated set, target at least 30% public-facing or experiential applications, 20% applications in which visual elements serve functional roles, 20% spatial or media-driven applications, 20% applications with playable mechanisms, and approximately 10% complex professional or administrative systems; categories may overlap.

2. Application Requirements

Each application must:

- define a realistic target user, purpose, and complete user goal;
- include multiple interacting controls, views, scenes, or content regions;
- contain at least five connected states with visible feedback;
- support a meaningful multi-step workflow ending in an observable result;
- include at least one validation, correction, retry, undo, or recovery path;
- preserve relevant state across navigation, retries, previews, or view changes;
- use local, simulated, or pre-populated data.

Visual properties such as color, motion, depth, scale, and spatial position should communicate hierarchy, progress, availability, validation, or state.

Avoid static landing pages, generic CRUD dashboards, isolated forms, simple calculators, and applications completed through a single click.

3. Diversity Requirements

Vary target users, purposes, workflows, interface structures, interaction methods, validation rules, recovery strategies, final outcomes, and visual styles. Do not repeat the same application concept with only cosmetic or industry-level changes.

4. Output Field Requirements

Use exactly the following five fields:

- `idx`: a sequential identifier beginning at 1;

- goal: the target user, workflow, recovery path, final outcome, and important state continuity;
- application specification: the complete webpage-generation instruction;
- domain: one of the eight predefined domains;
- style: "<Visual Style>", using one of the following styles: Editorial, Neo-brutalism, Glassmorphism, Retro-futurism/Y2K, Playful Toy-like, Cinematic Dark, Museum/Archive, Street Collage, Techwear/Tactical UI, Luxury Minimal, Chinese Fantasy, Swiss, Bauhaus-inspired, Craft/Scrapbook, Terminal/Monospace Ops, Infographic Newsroom, or Organic/Biophilic UI.

5. Final Output Constraints

- **Direct Output Only:** Return only a valid JSON array.
- **No Additional Text:** Do not include Markdown, comments, explanations, headings, or code fences.

```
[{
  "idx": 1,
  "goal": "Target user, workflow, recovery path, and outcome.",
  "application specification": "Complete webpage-generation instruction.",
  "domain": "One of the eight predefined domains.",
  "style": "<Visual Style>"
}]
```

—**TASK:** Generate the requested interactive web application specifications now.

Figure 12: Prompt used to generate diverse interactive web application specifications

System Instruction

Prompt of Web Application Generation

You are a professional front-end developer specializing in interactive, stateful, responsive, and visually polished web applications. Your task is to create a complete single-page web application that faithfully implements the provided application specification and visual style.

1. Input

Web application specification.

2. Functional Requirements

The application must:

- implement the complete workflow, interactions, state transitions, validation rules, recovery paths, and final outcome described in the specification;
- ensure that all controls and interactive elements function correctly and provide visible feedback;
- preserve relevant state across navigation, retries, previews, and view changes;
- use only local, simulated, or pre-populated data;
- avoid replacing required interactions with static mockups or non-functional placeholders.

3. Implementation Requirements

- Create one complete HTML document with embedded CSS and JavaScript.
- Use modern HTML5, CSS3, and vanilla JavaScript without external libraries or services.
- Use a clear semantic HTML structure.
- Ensure that the application runs directly in a modern web browser without additional setup.

4. Visual and Responsive Requirements

Follow the provided visual style consistently across typography, color, layout, controls, motion, and feedback. Maintain clear visual hierarchy, readable content, and consistent interaction behavior.

The application must render and function correctly at the standardized desktop viewport used for recording and evaluation. It must also adapt responsively to common mobile screen sizes without horizontal overflow, clipped content, overlapping elements, unreadable text, or unusable controls.

5. Final Output Constraints

- **Direct Output Only:** Return only the complete HTML document.

- **No Additional Text:** Do not include Markdown, explanations, introductory text, or code fences.
- **Self-Contained Output:** Include all required HTML, CSS, and JavaScript in the returned document.

—**TASK:** Generate the complete single-page HTML application now.

Figure 13: Prompt used to generate complete interactive HTML applications from the generated specifications

System Instruction

Failure Candidate Discovery Prompt

Role and Task

You are a professional QA auditor. Review the complete interaction video and identify potential webpage failures using the provided failure taxonomy.

1. Input

You will receive:

- a screen-recording video of a human operating the generated web application; and
- a taxonomy containing four dimensions, twelve subdimensions, and thirty failure modes.

Failure Taxonomy:

<FAILURE_TAXONOMY_TABLE>

2. Discovery Rules

Inspect user actions, interface responses, state changes, navigation, animation, and workflow outcomes.

Report only failures supported by visible evidence. Describe the observed action and interface behavior without inferring hidden causes or unperformed interactions.

Ignore recording artifacts, capture overlays, and operator skill. Merge observations belonging to the same continuous event.

3. Output Requirements

Return all identified events in chronological order using the following format:

<START_TIME>--<END_TIME>

<CONCISE_DESCRIPTION_OF_THE_OBSERVED_FAILURE>

Use MM:SS-MM:SS timestamps and one concise description for each event. Do not output failure-mode labels, explanations, headings, summaries, or unsupported events.

If no failure is visibly supported, output:

No visually supported failure found.

—**TASK:** Inspect the video and return all visually supported webpage failures now.

Figure 14: Prompt used to identify potential failures from interaction videos

System Instruction

Video–HTML Diagnostic Evaluation Prompt

Role and Task

You are a professional QA auditor responsible for diagnosing failures in AI-generated interactive web applications. For every supplied single-choice diagnostic question, compare all candidate failure modes and select exactly one option that is most directly supported by the available evidence and most relevant to the intended user workflow. Answer every question exactly once.

1. Input

You will receive:

1. a screen-recording video showing a human operating the generated web application;
2. the corresponding HTML/CSS/JavaScript source code; and
3. a set of diagnostic questions containing the question ID, target evaluation dimension, intended user workflow, question

text, and candidate failure modes with definitions.

2. Evidence Priority

Primary Evidence: Operation Video.

Use the video as the primary source for judging rendered appearance, layout, readability, motion, user actions, interaction responses, state changes, navigation, and workflow progression.

What the user visibly experiences takes precedence over what the source code appears intended to implement.

Supporting Evidence: HTML/CSS/JavaScript.

Use the source code to clarify ambiguous video evidence or verify displayed text, DOM structure, CSS properties, event handlers, state-management logic, asset references, and client-side behavior.

When the code conflicts with behavior clearly shown in the video, follow the video evidence.

3. Evaluation Rules

1. **Complete and dimension-specific evaluation.** Evaluate every question independently according to its target dimension, intended workflow, and candidate definitions. Do not consider unrelated defects.
2. **Internal candidate comparison.** Inspect every candidate before selecting an answer, but do not include the option-by-option comparison in the final output.
3. **Direct evidence only.** Select an option only when supported by observable video evidence or clear source-code evidence where the relevant video detail is ambiguous. Do not rely on speculation or assumed behavior.
4. **No unsupported interaction assumptions.** Do not infer the result of an action that is not performed in the video, unless the code verifies it unambiguously and implementation-level evidence is relevant to the question.
5. **Specific evidence.** Support the selected option with a concise description of the observed layout, interaction, animation, navigation, state, content, or workflow failure.
6. **Single-choice decision.** Select exactly one supplied `option_id`. If several options appear related, choose the one with the clearest evidence, most precise definition match, and greatest relevance to the intended workflow. Briefly acknowledge genuine ambiguity in the reasoning.
7. **Excluded considerations.** Do not evaluate recording quality, video compression, capture artifacts, operator skill, failures outside the target dimension, or failure types not included among the candidates.

4. Diagnostic Question Format

Each question follows this structure:

Question ID: <QUESTION_ID>

Question Type: `single_choice`

Target Evaluation Dimension:

<TARGET_EVALUATION_DIMENSION>

Generation Intent (User Workflow):

<GENERATION_INTENT_USER_WORKFLOW>

Question:

<QUESTION_TEXT>

Candidate Options:

- **A:** <OPTION_A_FAILURE_MODE>
Definition: <OPTION_A_DEFINITION>
- **B:** <OPTION_B_FAILURE_MODE>
Definition: <OPTION_B_DEFINITION>
- **C:** <OPTION_C_FAILURE_MODE>
Definition: <OPTION_C_DEFINITION>
- **D:** <OPTION_D_FAILURE_MODE>
Definition: <OPTION_D_DEFINITION>

5. Output Requirements

Return only one valid JSON object in the following form:

```
{
  "answers": [
    {
      "question_id": "<QUESTION_ID>",
```

```

    "reasoning": "<CONCISE_EVIDENCE_BASED_REASONING>",
    "selected_option_ids": ["A"],
    "evidence": {
      "option_id": "A",
      "video_description": "<BRIEF_DESCRIPTION_OF_THE_OBSERVED_EVIDENCE>"
    }
  }
]
}

```

The answers array must contain exactly one object for every supplied question_id. Each object must contain only question_id, reasoning, selected_option_ids, and evidence, in that order.

The selected_option_ids field must contain exactly one supplied option ID. The evidence object must contain only option_id and video_description.

Do not output unselected options, internal candidate comparisons, additional fields, summaries, Markdown code fences, or text outside the JSON object.

—TASK:

Evaluate every supplied diagnostic question and return the required JSON object now.

Figure 15: Prompt used for video–HTML diagnostic evaluation

System Instruction

Video-Only Diagnostic Evaluation Prompt

Role and Task

You are a professional QA auditor responsible for diagnosing failures in AI-generated interactive web applications. For every supplied single-choice diagnostic question, compare all candidate failure modes and select exactly one option that is most directly supported by the available video evidence and most relevant to the intended user workflow. Answer every question exactly once.

1. Input

You will receive:

1. a screen-recording video showing a human operating the generated web application; and
2. a set of diagnostic questions containing the question ID, target evaluation dimension, intended user workflow, question text, and candidate failure modes with definitions.

2. Evidence Priority

Primary Evidence: Operation Video.

Use the video as the sole source for judging rendered appearance, layout, readability, motion, user actions, interaction responses, state changes, navigation, and workflow progression.

Base all judgments on what is visibly rendered and experienced by the user in the video.

3. Evaluation Rules

1. **Complete and dimension-specific evaluation.** Evaluate every question independently according to its target dimension, intended workflow, and candidate definitions. Do not consider unrelated defects.
2. **Internal candidate comparison.** Inspect every candidate before selecting an answer, but do not include the option-by-option comparison in the final output.
3. **Direct evidence only.** Select an option only when it is supported by observable evidence in the video. Do not rely on speculation or assumed behavior.
4. **No unsupported interaction assumptions.** Do not infer the result of an action that is not performed or visibly demonstrated in the video.
5. **Specific evidence.** Support the selected option with a concise description of the observed layout, interaction, animation, navigation, state, content, or workflow failure.
6. **Single-choice decision.** Select exactly one supplied option_id. If several options appear related, choose the one with the clearest evidence, most precise definition match, and greatest relevance to the intended workflow. Briefly acknowledge genuine ambiguity in the reasoning.
7. **Excluded considerations.** Do not evaluate recording quality, video compression, capture artifacts, operator skill, failures outside the target dimension, or failure types not included among the candidates.

4. Diagnostic Question Format

Each question follows this structure:

Question ID: <QUESTION_ID>

Question Type: single_choice

Target Evaluation Dimension:

<TARGET_EVALUATION_DIMENSION>

Generation Intent (User Workflow):

<GENERATION_INTENT_USER_WORKFLOW>

Question:

<QUESTION_TEXT>

Candidate Options:

- **A:** <OPTION_A_FAILURE_MODE>
Definition: <OPTION_A_DEFINITION>
- **B:** <OPTION_B_FAILURE_MODE>
Definition: <OPTION_B_DEFINITION>
- **C:** <OPTION_C_FAILURE_MODE>
Definition: <OPTION_C_DEFINITION>
- **D:** <OPTION_D_FAILURE_MODE>
Definition: <OPTION_D_DEFINITION>

5. Output Requirements

Return only one valid JSON object in the following form:

```
{
  "answers": [
    {
      "question_id": "<QUESTION_ID>",
      "reasoning": "<CONCISE_EVIDENCE_BASED_REASONING>",
      "selected_option_ids": ["A"],
      "evidence": {
        "option_id": "A",
        "video_description": "<BRIEF_DESCRIPTION_OF_THE_OBSERVED_EVIDENCE>"
      }
    }
  ]
}
```

The answers array must contain exactly one object for every supplied question_id. Each object must contain only question_id, reasoning, selected_option_ids, and evidence, in that order.

The selected_option_ids field must contain exactly one supplied option ID. The evidence object must contain only option_id and video_description.

Do not output unselected options, internal candidate comparisons, additional fields, summaries, Markdown code fences, or text outside the JSON object.

—TASK:

Evaluate every supplied diagnostic question and return the required JSON object now.

Figure 16: Prompt used for video-only diagnostic evaluation

System Instruction

HTML-Only Diagnostic Evaluation Prompt

Role and Task

You are a professional QA auditor responsible for diagnosing failures in AI-generated interactive web applications from their HTML/CSS/JavaScript source code. For every supplied single-choice diagnostic question, compare all candidate failure modes and select exactly one option that is most directly supported by the available source-code evidence and most relevant to the intended user workflow. Answer every question exactly once.

1. Input

You will receive:

1. the complete HTML/CSS/JavaScript source code of the generated web application; and
2. a set of diagnostic questions containing the question ID, target evaluation dimension, intended user workflow, question text, and candidate failure modes with definitions.

2. Evidence Source

Only Evidence: HTML/CSS/JavaScript.

Use the complete source code as the only evidence for judging DOM structure, CSS properties, displayed text, asset references, event handlers, state variables, state-update logic, validation rules, navigation targets, and implemented client-side behavior.

No operation video, screenshot, rendered webpage, browser runtime, network log, or interaction trace is available.

The diagnostic question may mention a video, visual observation, user action, or timestamp because the same questions are used across input modalities. Interpret the underlying failure-diagnosis question using only the supplied source code. Do not claim that any visual or runtime behavior was directly observed.

Static source code may not conclusively establish browser-dependent, server-dependent, network-dependent, timing-dependent, or user-action-dependent behavior.

3. Evaluation Rules

1. **Complete and dimension-specific evaluation.** Evaluate every question independently according to its target dimension, intended workflow, and candidate definitions. Do not consider unrelated defects.
2. **Internal candidate comparison.** Inspect every candidate before selecting an answer, but do not include the option-by-option comparison in the final output.
3. **Direct source-code evidence only.** Select an option based on concrete evidence explicitly supported by the source code. When possible, identify the relevant element, selector, CSS property, text content, asset path, event handler, state update, validation condition, or navigation target.
4. **No runtime fabrication.** Do not invent rendered appearance, user actions, interaction results, animation behavior, navigation outcomes, network responses, or timestamps. Do not describe a possible implementation risk as a directly observed runtime failure.
5. **Specific evidence.** Support the selected option with a concise description of the relevant HTML/CSS/JavaScript implementation evidence.
6. **Single-choice decision.** Select exactly one supplied `option_id`. If several options appear related, choose the one with the strongest explicit source-code evidence, most precise definition match, and greatest relevance to the intended user workflow.
If no option is conclusively established by static source code, select the option with the strongest available source-code support. This selection represents relative evidential support rather than confirmation of an observed runtime failure.
7. **Excluded considerations.** Do not evaluate unavailable visual or runtime evidence, failures outside the target dimension, or failure types not included among the candidates.

4. Diagnostic Question Format

Each question follows this structure:

Question ID: <QUESTION_ID>

Question Type: `single_choice`

Target Evaluation Dimension:

<TARGET_EVALUATION_DIMENSION>

Generation Intent (User Workflow):

<GENERATION_INTENT_USER_WORKFLOW>

Question:

<QUESTION_TEXT>

Candidate Options:

- **A:** <OPTION_A_FAILURE_MODE>
Definition: <OPTION_A_DEFINITION>
- **B:** <OPTION_B_FAILURE_MODE>
Definition: <OPTION_B_DEFINITION>
- **C:** <OPTION_C_FAILURE_MODE>
Definition: <OPTION_C_DEFINITION>

- **D:** <OPTION_D_FAILURE_MODE>
Definition: <OPTION_D_DEFINITION>

5. Output Requirements

Return only one valid JSON object in the following form:

```
{
  "answers": [
    {
      "question_id": "<QUESTION_ID>",
      "reasoning": "<CONCISE_EVIDENCE_BASED_REASONING>",
      "selected_option_ids": ["A"],
      "evidence": {
        "option_id": "A",
        "video_description": "<BRIEF_DESCRIPTION_OF_THE_SOURCE_CODE_EVIDENCE>"
      }
    }
  ]
}
```

The `answers` array must contain exactly one object for every supplied `question_id`. Each object must contain only `question_id`, `reasoning`, `selected_option_ids`, and `evidence`, in that order.

The `reasoning` field must explain why the selected option has the strongest explicit HTML/CSS/JavaScript support without claiming that the failure was visually or operationally observed.

The `selected_option_ids` field must contain exactly one supplied option ID. The `evidence` object must contain only `option_id` and `video_description`.

The `video_description` field is retained for output compatibility but must contain a concise description of the source-code evidence rather than video evidence.

Do not output unselected options, internal candidate comparisons, additional fields, summaries, Markdown code fences, or text outside the JSON object.

—TASK:

Evaluate every supplied diagnostic question and return the required JSON object now.

Figure 17: Prompt used for HTML-only diagnostic evaluation

System Instruction

V2Lens Video Analyst Prompt

Role and Task

*You are the **Video Analyst** in V2Lens. Examine the complete interaction video and answer each single-choice diagnostic question.*

For every question:

- determine whether each candidate failure mode is `present` or `absent`;
- provide concise video evidence for each option; and
- select the single option that most directly explains the observed failure.

Several failure modes may be visible, but the initial answer must be the one most relevant to the intended user workflow.

1. Evidence Rules

Use only directly observable user actions, interface responses, state changes, navigation behavior, and workflow outcomes from the video.

Ignore recording quality, compression artifacts, operator skill, and recording-tool overlays. Do not infer behavior from interactions that are not shown.

2. Input

Video ID: <VIDEO_ID>

Question and Intended Workflow:

<QUESTION_AND_WORKFLOW>

Candidate Options and Definitions:

<CANDIDATE_OPTIONS_AND_DEFINITIONS>

3. Output Requirements

Return only one valid JSON object:

```
{
  "question_id": "<QUESTION_ID>",
  "initial_answer": "A",
  "reasoning": "<CONCISE_VIDEO_BASED_REASONING>",
  "option_assessments": [
    {
      "option_id": "A",
      "verdict": "present|absent",
      "video_evidence": "<OBSERVED_INTERACTION_EVIDENCE>"
    }
  ]
}
```

Include one assessment for every candidate option. Do not output Markdown or additional text.

—TASK:

Analyze the video and return the required JSON object now.

Figure 18: Prompt used by the V2Lens Video Analyst

System Instruction**V2Lens HTML Verifier Prompt****### Role and Task**

*You are the **HTML Verifier** in V2Lens. Inspect the webpage's HTML/CSS/JavaScript source and evaluate the implementation evidence for every candidate failure mode.*

For each option, determine whether the source code:

- supports the diagnosis;
- contradicts the diagnosis; or
- is inconclusive.

1. Evidence Rules

Inspect relevant DOM structure, CSS rules, text, assets, event handlers, validation conditions, navigation targets, and state-update logic.

Use only explicit source-code evidence. Code findings provide implementation context but do not independently prove that a failure appeared during the recorded interaction.

2. Input**Question and Intended Workflow:**

<QUESTION_AND_WORKFLOW>

Candidate Options and Definitions:

<CANDIDATE_OPTIONS_AND_DEFINITIONS>

Video Analyst Assessments:

<VIDEO_ANALYST_ASSESSMENTS>

HTML/CSS/JavaScript Source:

<WEBPAGE_SOURCE>

3. Output Requirements

Return only one valid JSON object:

```
{
  "question_id": "<QUESTION_ID>",
  "findings": [
    {
      "option_id": "A",
```



```

    "code_verdict": "supports|contradicts|inconclusive",
    "code_evidence": "<RELEVANT_IMPLEMENTATION_EVIDENCE>"
  }
]
}

```

Include one finding for every candidate option. Do not output Markdown or additional text.

—TASK:

Inspect the source code and return the required JSON object now.

Figure 19: Prompt used by the V2Lens HTML Verifier

System Instruction

V2Lens Evidence Judge Prompt

Role and Task

*You are the **Evidence Judge** in V2Lens. Integrate Video Analyst assessments, HTML Verifier findings, and available video and source-code evidence. Retain the initial answer unless another candidate is more strongly supported by the collected evidence. When a stronger alternative exists, produce a challenge answer and the evidence supporting it.*

1. Evidence Rules

The interaction video is the primary evidence, while source code provides supporting implementation evidence.

When necessary, use frame extraction, crop and zoom, visual measurement, or HTML inspection to resolve insufficient or conflicting evidence.

A challenge must be supported by direct observable behavior and must not rely only on indirect source-code clues.

2. Input

Question, Workflow, and Candidate Options:

<QUESTION_WORKFLOW_AND_OPTIONS>

Initial Answer:

<VIDEO_ANALYST_INITIAL_ANSWER>

Video Analyst Assessments:

<VIDEO_ANALYST_ASSESSMENTS>

HTML Verifier Findings:

<HTML_VERIFIER_FINDINGS>

Interaction Video and Webpage Source:

<VIDEO_AND_SOURCE_CONTEXT>

3. Output Requirements

Return only one valid JSON object:

```

{
  "question_id": "<QUESTION_ID>",
  "decision": "retain|challenge",
  "challenge_answer": "<OPTION_ID_OR_EMPTY>",
  "reasoning": "<CONCISE_CROSS_MODAL_REASONING>",
  "supporting_evidence": "<DIRECT_EVIDENCE_FOR_THE_DECISION>"
}

```

Use retain and an empty challenge_answer when no alternative is more strongly supported. Do not output Markdown or additional text.

—TASK:

Evaluate the collected evidence and return the required JSON object now.

Figure 20: Prompt used by the V2Lens Evidence Judge

System Instruction

V2Lens Reviewer Prompt

Role and Task

*You are the independent **Reviewer** in V2Lens. Compare two anonymized candidate answers using the original interaction video, webpage source, diagnostic question, and candidate definitions. You are not told which candidate is the initial answer or the challenge answer. You cannot access the Evidence Judge's reasoning or supporting evidence. Apply the same standard to both candidates and determine which is more directly supported.*

1. Evidence Rules

Use the interaction video as the primary evidence and source code as supporting evidence. Inspect additional frames, visual details, measurements, or source code when necessary.

Do not prefer a candidate because it was routed for review. Judge both independently.

2. Input

Question, Workflow, and Candidate Definitions:

<QUESTION_WORKFLOW_AND_DEFINITIONS>

Candidate 1:

<ANONYMIZED_CANDIDATE_1>

Candidate 2:

<ANONYMIZED_CANDIDATE_2>

Interaction Video and Webpage Source:

<VIDEO_AND_SOURCE_CONTEXT>

3. Output Requirements

Return only one valid JSON object:

```
{
  "question_id": "<QUESTION_ID>",
  "preferred_candidate": "candidate_1|candidate_2|undetermined",
  "reasoning": "<CONCISE_INDEPENDENT_EVIDENCE_BASED_REASONING>"
}
```

Use undetermined when the evidence does not reliably distinguish the two candidates. Do not output Markdown or additional text.

—TASK:

Independently review the two candidates and return the required JSON object now.

Figure 21: Prompt used by the V2Lens Reviewer